

## Detailed Explanation

### Function Definition and Documentation

Function Name: 'decimalToBinary'

#### Parameters:

int '\$decimal': The input decimal number to be converted.

#### Return Value:

A string representing the binary equivalent of the input decimal number.

#### Exceptions:

Throws 'InvalidArgumentException' if the input is not a valid integer or is negative.

### Logic Breakdown

#### Input Validation:

Special case: If the input is zero, the function immediately returns '0'.

### Binary Conversion Using Bitwise Operators:

An empty string '\$binary' is initialized to build the binary representation.

The 'while' loop continues as long as the decimal number is greater than zero:

Bitwise AND Operation (&): The expression '(\$decimal & 1)' extracts the least significant bit of the decimal number.

String Concatenation: The extracted bit is appended to the left side of the '\$binary' string to build the binary number from right to left

Right Shift Operation (>>): The decimal number is shifted one bit to the right, effectively dividing it by 2 and discarding the least significant bit.

### Return Statement:

After the loop completes, the function returns the constructed binary string.

### Time Complexity Analysis

Initialization and Input Validation: The checks and initializations are constant-time operations,  $O(1)$ .

Binary Conversion Loop: The 'while' loop iterates approximately ' $\log_2(n)$ ' times, where 'n' is the value of the decimal number. Each iteration involves constant-time operations (bitwise AND, string concatenation, and bitwise shift), making the time complexity  $O(\log n)$ .

### Space Complexity Analysis

Space Complexity: The function uses a fixed amount of additional space for variables, and the length of the output string is proportional to the number of bits required to represent the decimal number. Therefore, the space complexity is  $O(\log n)$ .

## Bitwise Operations in 'decimalToBinary'

Bitwise operations are fundamental low-level operations that directly manipulate bits of binary representations of integers. In the 'decimalToBinary' function, two bitwise operations are primarily used: the bitwise AND ('&') and the right shift ('>>'). Below is an in-depth explanation of these operations and how they are applied in the function.

### Bitwise AND ('&')

#### Operation:

The bitwise AND operation takes two binary operands and performs a logical AND on each pair of corresponding bits. The result is a binary number where each bit is set to 1 if and only if both corresponding bits of the operands are 1; otherwise, the bit is set to 0.

#### Usage in 'decimalToBinary':

```
$binary = ($decimal & 1) . $binary;
```

#### Purpose:

The expression '(\$decimal & 1)' extracts the least significant bit (LSB) of the decimal number. This operation checks whether the LSB is 0 or 1, effectively determining if the number is even or odd.

#### Example:

Consider 'decimal = 5' (binary: '101'):

'5 & 1' results in '101 & 001' which equals '001' (binary) or '1' (decimal).

This result tells us that the LSB of 5 is 1, so we append '1' to the binary string.

## Right Shift ('>>')

### Operation:

The right shift operation takes a binary number and shifts all its bits to the right by a specified number of positions. Each shift to the right effectively divides the number by 2, discarding the fractional part (in the context of integers, this means discarding the least significant bit).

Usage in 'decimalToBinary':

```
$decimal = $decimal >> 1;
```

### Purpose:

The right shift operation is used to reduce the decimal number by a factor of 2, effectively moving to the next bit for extraction in the next iteration of the loop.

### Example:

Consider decimal = 5 (binary: 101):

5 >> 1 results in 101 >> 1 which equals 10 (binary) or 2 (decimal).

After shifting right, the number becomes 2, allowing the loop to continue extracting the next bit.

# Detailed Breakdown of the Function

## Function Definition and Input Handling

```
function decimalToBinary(int $decimal): string
{
    if ($decimal === 0) {
        return '0';
    }
}
```

Initialization: We handle the special case where the input is 0 by returning '0' immediately.

## Core Loop for Bitwise Conversion

```
$binary = "";
while ($decimal > 0) {
    $binary = ($decimal & 1) . $binary;
    $decimal = $decimal >> 1;
}
return $binary;
}
```

Initialization: We initialize an empty string '\$binary' to build the binary representation.

Loop: The loop continues until the decimal number is reduced to 0.

Bitwise AND: ( $\$decimal \& 1$ ) extracts the LSB and appends it to the left side of the \$binary string.

Right Shift:  $\$decimal = \$decimal \gg 1$  shifts the decimal number to the right, preparing it for the next iteration.

## Time Complexity Analysis

Time Complexity: The loop iterates approximately  $\log_2(n)$  times, where  $n$  is the value of the decimal number. Each iteration involves constant-time operations, resulting in a time complexity of  $O(\log n)$ .